

HelixQA

HelixQA

アンチブラフQAオーケストレーション — すべての**PASS**に、実際のユーザーが機能を利用できる証拠を伴う、自律型クロスプラットフォームセッション

概要

HelixQAは、クロスプラットフォームテスト（Android、Android TV、Web、デスクトップ）向けのアンチブラフQAオーケストレーションフレームワークであり、YAMLテストバンク、リアルタイムクラッシュ検出、ステップごとの証拠キャプチャ、LLMプラスコンピュータビジョンによる自律型QAセッションを組み合わせ、機能がエンドツーエンドで真に動作することを証明する。これはConstitutionの必須QAテストタイプ（§11.4.169）である。

短い説明

アンチブラフQAオーケストレーター（Go）は、記述されたテストバンクと完全自律型のLLMおよびビジョン駆動QAセッションをプラットフォーム間で実行し、クラッシュを検出し、各ステップをキャプチャされた証拠（スクリーンショット、logcat、動画、スタックトレース）と照合し、AI修正パイプライン向けに証拠豊富なチケットを自動生成する。

長い説明

HelixQAは、Goフレームワークであり、その唯一無二の設計の中心はConstitutionの§11.4運用ルールにある。すなわち、リリースの基準は「テストがパスすること」ではなく「ユーザーが機能を利用できること」であり、出力されるすべてのPASSには実行中にキャプチャされた実証的な証拠が伴わなければならない。証拠がなければグリーンライトは出ず、例外は認められない。本フレームワークは、スクリプト化されたテストと未知のテストの両方をカバーする二つの相補的なモードで動作する。

第一に、記述されたテストバンク — プラットフォームターゲティング、優先度、順序付きステップ（名称/アクション/期待値）、タグ、ドキュメント参照を備えたTC-XXXケース群であるYAMLスイートを実行する。各ステップの検証、リアルタイムのクラッシュ/ANR検出（AndroidはADB、Web/デスクトップはプロセス監視）、集中型証拠収集、そして下流のAI修正パイプライン向けに整形済みのMarkdownチケットを自動生成する。

第二に、完全自律型**QA**セッション — アプリをLLM搭載エージェントとコンピュータビジョンに委ね、無人で動作させる。このセッションは四つの規律あるフェーズで構成される。セットアップ（LLMの選択、プロジェクトドキュメントからの機能マップ作成、CLIエージェントの起動、ビジョンエンジンの初期化）、ドキュメントに基づく検証（すべてのドキュメント化された機能を検証）、好奇心駆動型の探索（エッジケースや非ドキュメント化された動作を意図的に試行）、そしてMarkdown/HTML/JSON形式でのレポートとクリーンアップ（すべての発見事項は動画タイムスタンプ付きの証拠にリンクされる）。

重要なのは、HelixQAは自己評価を行わない点である。四つの外部Goサブモジュール（LLMsVerifier、LLMOrchestrator、VisionEngine、DocProcessor）を統合し、challengesおよびcontainersの共有インフラを再利用するため、アプリを操作するコンポーネントと、それが正常に動作したかを判断するコンポーネントは別々である。自身のスイートも、make anti-bluff（静的スキャン+動作アンカーマニフェスト+ミュートセッションラチェット）や、組み込みS1.1ミュートーションを持つ8フェーズオーケストレーターChallengeを通じて、他のテスト対象と同じ基準で評価される。15行のテストタイプカバレッジマトリックスにより、すべての謳われた機能は具体的な実行可能アセットと特定の証拠形態に紐付けられ、フレームワーク自身の主張も、テスト対象製品に下す評価と同様に、証拠に基づくものとなる。

コンテンツ

なぜこれを作ったのか

従来のQAは「アサーションが通った」という結果をもって合格としていた。しかし、この方法では、Constitutionが「ブラフ」と呼ぶ種類の失敗が見逃されてしまう——機能は「動作報告」されているが、実際のユーザーにとっては壊れているという状態だ。HelixQAは、この問題をQAの段階で根本的に排除するために作られた。実行時の物的証拠（スクリーンショット、logcat、動画、スタックトレース、レポート）がなければPASSと評価せず、証拠のない「グリーン」のサマリーラインは、機能が欠落しているのと同等の重大な欠陥として扱う。また、複数のプラットフォームにわたる包括的な手動QAがスケールしないという労力の問題も解決する。完全に自律的なセッションを実現することで、人的リソースを必要としない。

なぜゲームチェンジャーなのか

HelixQAは、ほとんど同じツールに共存することのない二つの要素を融合させた。厳格なエビデンスに基づくQAゲーティングと、自律的な自己駆動型探索だ。LLMにビジョン機能を組み合わせたエージェントは、実際のアプリを開き、ドキュメント化された機能をすべて検証し、誰もテストを書いていない未知のバグを探し出す。そして、その過程で裁判に耐える証拠のトレイルを生成する——「テストした」という言葉の代わりに、「これが動画、これがlogcat、これがチケット」という具体的な証拠が提示される。さらに、Constitutionで定義されたQAサブモジュールであるため、これを導入することで、一つのチームだけでなく、関連するすべてのプロダクトのQAの信頼性が一気に底上げされる。

革新的なポイント

- **ブラフ防止エビデンス契約** — すべてのチェックのPASSは、実行時に取得した証拠と紐付けられる。CIのグリーンラインは必要条件ではあるが十分条件ではなく、証拠のないグリーンのサマリーは、機能欠落と同等の重大な欠陥として評価される。
- **自律的なドキュメント駆動型+好奇心駆動型探索** — ドキュメント化された機能をすべて検証した後、オフスクリプトで動作し、実際のユーザーが遭遇するエッジケース（空の入力、急速な操作、未ドキュメントのパス）を探索する。手動で書かれたテストスイートでは予測できない領域だ。
- **ビジョンオラクル** — GoCVの機械ビジョンとLLM Vision APIが、画面上のUIを実際に視認し、トークンやプロパティレベルのアサーションでは見逃されがちな、視覚的に壊れた状態を検出する。
- **構造ベースのテストバンク** — バンクの文字列は構造を記述し、実行時にLLMが生成する質問プロンプトを駆動する（CONST-046）。そのため、UIテキストが翻訳されても、一つのバンクが複数のロケールで機能する。
- **AI修正パイプライン向けチケット** — 自動生成されるMarkdown形式のIssueには、証拠バンドルが添付されており、人間のトリアーガーを介さずに、そのまま下流の修正エージェントに渡すことができる。

すべてのプロダクトでどのように使われるか（与えられる力）

必須の品質基盤として（Constitution §11.4.169ではhelix_qaサブモジュールが必須のテストタイプの一つに指定されている）、HelixQAは、関連するすべてのプロダクトに以下の力を与える：

- **自律QAセッション** —
`helixqa autonomous --project ... --platforms android,desktop,web`
という単一のコマンドで、LLMとビジョン機能を組み合わせたエージェントが実際のアプリを無人で駆動し、カバレッジ目標に向けて動作

する。その間、レポート、チケット、動画が自動生成され、人間の介在は不要。

- テストバンク／スイート — YAMLバンク（ラウンド219、フロア30以上）、プラットフォームごとにターゲットを絞り、優先順位を付け、検証するドキュメントと行単位でトレース可能。
- 取得済みエビデンス — スクリーンショット、logcat、動画、スタックトレース、完全なタイムラインが一元管理され、すべてのレポートからリンクされている。そのため、後からどの判定も再生・監査が可能。
- 独立した判定（§11.4.141独立原則） — LLMが駆動するissuedetectorとビジョンオラクルは、アプリの動作を、それを操作したエージェントから独立して判定する。これにより、システムが自分の作業を正しいと自己評価するという古典的な失敗を構造的に排除。
- ゲート+ミューテーションラチェット — make qa-all / make anti-bluffおよびchallenges/scripts/helixqa_orchestrator_challenge.sh（8フェーズ、§1.1のミューテーション機能内蔵）により、HelixQA自体の信頼性が常に検証される。さらに、締め切りのプレッシャーがあっても--skip-helixqaのような逃げ道をあえて用意していないため、規律が緩むことはない。

コンテンツ

最大の技術的課題とその解決方法

- QA自体の誤検出防止 — ブラフを検出するツールがブラフになってはならない → すべてのステップは収集されたエビデンスに対して検証され、エビデンスのないPASSはPASSではなく欠陥としてスコアリングされ、動作アンカーマニフェスト（CONST-035）により、各宣伝された機能は実行可能なテストに紐づけられ、機能を主張するには必ずそれを実行するものが必要となる。
- 一つの頭脳で異種プラットフォームを制御 — Android、Android TV、Web、デスクトップは入力モデルを共有しない → 単一のnavigatorパッケージがプラットフォーム固有のActionExecutor（ADB、Playwright、X11）とプラットフォームごとのクラッシュ検出器（Android/Web/デスクトップ）を抽象化し、オーケストレーションロジックは一度書かれ、プラットフォームの違いは端に留まる。
- 自律エージェントを有用に、混沌とさせない — 監視されないLLMがアプリ内を永遠に彷徨う可能性 → LLMsVerifierが適切なモデルをスコアリング・選択し、LLMOrchestratorがヘッドレスのCLIEージェント（opencode、claude-code、gemini、junie、qwen-code）を管理、DocProcessorが探索の目標を与えるフィーチャーマップを構築、VisionEngineがすべての判断をモデルの想像ではなく画面上の実際のピクセルに基づかせる。
- ローカライズに対応したテストバンク — 英語のUIテキストをハードコーディングしたスイートは15の言語で壊れる → バンクは構造のみを記述し、ユーザー向けのプロンプトテキストはLLM/リソースから実行時に読み込まれる（CONST-046）ため、同じバンクがロケールに関係なく同じ動作を検証する。
- ゲートが偽物でないことの証明 — それ自体が失敗しないアンチブラフゲートは究極のブラフ → 対になった§1.1のミューテーションが型のエビデンスキャプチャやアンチブラフアサーションを削除し、ゲートに

FAILを要求し、ミューテーションラチェットがその保証が時間とともに密かに損なわれないようにする。

テックスタック

- **Go 1.24+** オークストレーター — 理由: QAは製品が動作するあらゆる場所で実行されなければならないため、単一のスタティックリンクされた高速でポータブルなバイナリが、ランタイム負荷の大きい代替手段に勝る。方法: cmd/helixqaのCLIが、run/list/report/autonomous/versionといったコンポーザブルなサブコマンドを提供。
- **YAML** テストバンク (**pkg/testbank**) — 理由: スイートは宣言的で読みやすく、Goに触れずに人間が編集可能であるべき。方法: version/name/test_cases[] (id、category、priority、platforms、順序付きsteps[]、documentation_refs[]) で、機能ドキュメントへのトレサビリティを確保。
- クラッシュ/**ANR**検出器 (**pkg/detector**) — 理由: 最も重要な失敗は、事後的なアサーションではなく、インタラクション中にリアルタイムで発生するもの。方法: AndroidではADB (pidof/logcat/screencap)、Web/デスクトップではpgrepを使用し、テストがドライブしている間にプロセスを監視。
- エビデンス収集 (**pkg/evidence**、**pkg/session**) — 理由: アンチブラフ契約は、すべてのPASSが物理的な証拠に裏付けられて初めて現実のものとなる。方法: スクリーンショット、logcat、動画、スタックトレースをSessionRecorderのタイムラインにキャプチャし、すべてのレポートがそれにリンクされる。
- 自律セッション (**pkg/autonomous**、**pkg/navigator**、**pkg/issuedetector**) — 理由: 4つのプラットフォームにわたる包括的な手動QAはスケールしないため、探索自体が自律的に動作する必要がある。方法: 4フェーズのSessionCoordinatorに加え、ActionExecutor (ADB/Playwright/X11) と、視覚、UX、アクセシビリティ、機能的な欠陥をカバーするLLMバグ検出。
- 外部サブモジュール — 理由: 再利用性と疎結合 (CONST-051)、そして何よりもナビゲーターとジャッジの分離。方法: LLMsVerifier (モデルスコアリング)、LLMOrchestrator (ヘッドレスCLIエージェント)、VisionEngine (GoCV + LLM Vision)、DocProcessor (フィーチャマップ/カバレッジ)、それぞれが独立したコンポーネントとして所有。
- アンチブラフゲート + ミューテーションラチェット — 理由: HelixQAに対して、それが他のすべてに強制するのと同じ§1.1の契約を遵守させるため。方法: make anti-bluffスキャンに加え、動作アンカーマニフェストとミューテーションラチェット、そして helixqa_orchestrator_challenge.shを8フェーズのエンドツーエンドバリデータとして使用。
- **15行のカバレッジマトリックス (docs/test-coverage.md)** — 理由: CONST-050(B)により、ギャップのない完全に説明可能なテストタイプセットが義務付けられている。方法: 各行は具体的な実行可能アセットと特定の収集エビデンスの形に紐づけられ、カバレッジは主張ではなく検証可能な事実となる。

ステータスと誠実性に関する注意事項

- ステータス: ベータ版。現在も開発中 (READMEのステータスバナーは第219版)。独自の「ブラフ防止基準」に準拠。
- ライセンス: **Apache-2.0**。インストール方法: `go install digital.vasic.helixqa/cmd/helixqa@latest`。

優先度レベル: Helix-プライマリ — Helixファミリーにおける品質・ブラフ防止の必須基盤。機能が実際に機能することを検証するための柱。